CS6370: Natural Language Processing
Assignment 1 Report (Part-A)

Release Date: 16th February 2026 **Total Marks:** 25

Name: Vignesh Natarajkumar
Roll No.: EE23B087

Part 1: Sentence Segmentation

1. **Question:** Suggest a simplistic top-down approach to sentence segmentation for English texts. Do you foresee issues with your proposed approach in specific situations? Provide supporting examples and possible strategies that can be adopted to handle these issues. [2 marks]

   Answer: In order to split sentences from a corpus, I vaguely specify a sentence as a string of words with some atomic meaning. Usually, these meanings are separated from each other by punctuations, and hence the top-down approach to divide sentences is to look at specified punctuation patterns.

   A simple set of rules I propose and implement are:

   - The sentence ends with one of these punctuations - [., !, ?]
   - A space follows the punctuation mark that ends a sentence
   - The space is followed by a number or a capitalized alphabet.

   This will work for common sentences, and is designed to avoid some problems in sentence segmentation I foresaw, which are:

   - Decimals, such as `3.14`
   - URLs and E-mail IDs, such as `yourname.com`
   - Consecutive punctuation, such as `...`
   - Abbreviations, such as `U.S.A.`
   - Punctuation mid-sentence, such as `the apples etc.  had ripened`.

   However, there are still cases where the proposed set of rules will incorrectly segment sentences:

   - Strings like `Dr.  Smith` and `U.S.A. Today`.
   - Quotations inside sentences, like `He said "Hi!  How are you?"`

   To tackle this, one could introduce some approaches, such as an exception list or lookup dictionary of common abbreviations (e.g., Mr., Dr., Inc., e.g.) that instructs the segmenter to ignore the rule when these tokens are encountered. We could also expand the rule to allow for an optional layer of closing punctuation (like ”, ””, or)) between the primary punctuation mark and the space.

However, relying solely on top-down, hand-crafted rules without understanding the nature and frequency of errors within the target corpus is inefficient, because this task is ill-defined and context-dependent. A rule-set made to address every corner case would be unmanageable and bloated.

Question: NLTK is one of the most commonly used packages for Natural Language Processing. What does the Punkt Sentence Segmenter in NLTK do differently from a simple top-down (rule-based) sentence segmentation approach? Additionally, compare both of the above approaches with spaCy's sentence segmentation method. [1 mark]

Answer: The Punkt method is an unsupervised learning approach to end sentences. It identifies the probability with which a particular pattern ends a sentence, and so the probability of abbreviations such as Mr. will be lower than that of other words that end with a full stop.

Since Punkt relies on the distributional structure of the corpus on which it was trained, whose weights are loaded in the code, a change in the distribution of the evaluation set will lead to poor results. For example, if I do not capitalize the first letter of every sentence, then Punkt would fail, because it has not seen this pattern before.

However, Punkt is very useful for localized use-cases, such as parsing just a corpus of legal or educational documents.

spaCy uses a supervised deep learning model based on dependency parsing to identify sentences [4]. As per [7], a sentence is "a directed graph of words connected by binary, typed grammatical relationships". This means that the model learns the relationships between words from a corpus in the form of a logical tree, and a sentence is basically a complete grammatical tree with words and word tags.

Since the focus is no longer on purely structural details, this method is robust to variations in typing across datasets and is very useful when trained on a general-purpose, large-scale corpus for the target language, which is what the code loads.

There are several situations that would lead to a low-confidence prediction from spaCy simply because the training data might not have encountered such patterns. For example, sentences with wrong capitalization, no verbs being used, and lists, such as `Grocery list: 1. Apples 2. Bananas 3. Carrots`, which will produce 4 sentences when it is actually just one. Further, the root node of this dependency parsing is usually a verb, and a sentence with multiple seemingly independent verbs might cause spaCy to struggle.

The naive rule-based approach thus underperforms in comparison to both the unsupervised learning approach of Punkt and the dependency parsing of spaCy, though the effectiveness of the learning-based methods depends on the corpus used for training and the patterns the model has learnt.

2. **Question:** Design an adversarial test suite for sentence segmentation: [4 marks]
   (a) Create 15 sentences that are likely to break a naive segmentation system.
   (b) Categorize them by error type (e.g., abbreviations, decimals, ellipsis, quotes, multiple punctuation, etc.).

(c) Evaluate how many errors each segmenter makes on your test suite.

Answer:

(a): The adversarial test suite is given below:

1. `Dr. J. R. R. Tolkien, Jr. was an author.`
2. `He yelled, "Stop right there!" Then he ran.`
3. `i went to the store. they were out of milk. so i left.`
4. `Let's meet on Oct. 14. 2026 is going to be a great year.`
5. `Grocery list: 1. Apples. 2. Bananas. 3. Carrots.`
6. `The item costs Rs. 50. 20 is the maximum discount.`
7. `while (x < 5.) { x++; }. That is the loop.`
8. `Have you read "Do Androids Dream of Electric Sheep?" Today, it is considered a classic.`
9. `I am so happy today!!! ... :-) Are you coming?`
10. `Prof. A. B. Smith (b. 1920) lived in Washington, D.C. His entire life.`
11. `She whispered... "Who is there?"`
12. `Wait, what?! Why are we doing this?!`
13. `Just sitting here. In the dark. Alone.`
14. `I bought apples, oranges, etc. They were very fresh.`
15. `And then I was like omg and he was like no way and I ran out.`

(b): The categorization of the sentences in the test suite above is:

- Abbreviations, Chained abbreviations, capitalized Pronouns, and Initials: **Sentences 1, 10, 14**.
- Quotations and Edge-Case Punctuation: **Sentences 2, 8, 11**.
- Unconventional capitalization: **Sentence 3**.
- Decimals, Dates, Abbreviations, Currency, and Numbers: **Sentences 4, 6**.
- Fragments, Lists, and massive sentences with incorrect punctuation and verb usage: **Sentences 5, 13, 15**.
- Code Syntax and Symbols: **Sentences 7, 9**.
- Multiple/Consecutive Punctuation: **Sentence 12**.

(c): Upon running each of these sentences through the 3 sentence segmenters, we get the following results:

- **Naive Segmenter:** Score: 6/15 (9 errors). This method failed on the following sentences:
  - **Input:** `Dr. J. R. R. Tolkien, Jr. was an author.`
    Output: `['Dr.', 'J.', 'R.', 'R.', 'Tolkien, Jr. was an author.']`
  - **Input:** `He yelled, "Stop right there!" Then he ran.`
    Output: `['He yelled, "Stop right there!" Then he ran.']`
  - **Input:** `i went to the store. they were out of milk. so i left.`
    Output: `['i went to the store. they were out of milk. so i left.']`
  - **Input:** `Let's meet on Oct. 14. 2026 is going to be a great year.`
    Output: `["Let's meet on Oct.", '14.', '2026 is going to be a great year.']`
  - **Input:** `Grocery list: 1. Apples. 2. Bananas. 3. Carrots.`
    Output: `['Grocery list: 1.', 'Apples.', '2.', 'Bananas.', '3.', 'Carrots.']`
  - **Input:** `The item costs Rs. 50. 20 is the maximum discount.`
    Output: `['The item costs Rs.', '50.', '20 is the maximum discount.']`
  - **Input:** `Have you read "Do Androids Dream of Electric Sheep?" Today, it is considered a classic.`
    Output: `['Have you read "Do Androids Dream of Electric Sheep?" Today, it is considered a classic.']`
  - **Input:** `I am so happy today!!! ... :-) Are you coming?`
    Output: `['I am so happy today!!! ... :-) Are you coming?']`
  - **Input:** `Prof. A. B. Smith (b. 1920) lived in Washington, D.C. His entire life.`
    Output: `['Prof.', 'A.', 'B.', 'Smith (b.', '1920) lived in Washington, D.C.', 'His entire life.']`
- **NLTK Punkt Segmenter:** Score: 9/15 (6 errors). This method failed on the following sentences:
  - **Input:** `Grocery list: 1. Apples. 2. Bananas. 3. Carrots.`
    Output: `['Grocery list: 1.', 'Apples.', '2.', 'Bananas.', '3.', 'Carrots.']`
  - **Input:** `The item costs Rs. 50. 20 is the maximum discount.`
    Output: `['The item costs Rs.', '50.', '20 is the maximum discount.']`
  - **Input:** `while (x < 5.) { x++; }. That is the loop.`
    Output: `['while (x < 5.)', '{ x++; }.', 'That is the loop.']`
  - **Input:** `I am so happy today!!! ... :-) Are you coming?`
    Output: `['I am so happy today!!!', '... :-) Are you coming?']`
  - **Input:** `Prof. A. B. Smith (b. 1920) lived in Washington, D.C. His entire life.`
    Output: `['Prof. A.', 'B. Smith (b.', '1920) lived in Washington, D.C. His entire life.']`

- **Input:** `Wait, what?!  Why are we doing this?!`
  Output: `['Wait, what?!', 'Why are we doing this?', '!']`

- **spaCy Segmenter:** Score: 10/15 (5 errors). This method failed on the following sentences:
  - **Input:** `Grocery list:  1.  Apples.  2.  Bananas.  3.  Carrots.`
    Output: `['Grocery list:', '1.  Apples.', '2.', 'Bananas.', '3.  Carrots.']`
  - **Input:** `The item costs Rs.  50.  20 is the maximum discount.`
    Output: `['The item costs Rs.  50.  20 is the maximum discount.']`
  - **Input:** `while (x < 5.)  { x++; }.  That is the loop.`
    Output: `['while (x < 5.)', '{ x++; }.', 'That is the loop.']`
  - **Input:** `Prof.  A. B. Smith (b.  1920) lived in Washington, D.C. His entire life.`
    Output: `['Prof.  A. B. Smith (b.  1920) lived in Washington, D.C.', 'His entire life.']`
  - **Input:** `And then I was like omg and he was like no way and I ran out.`
    Output: `['And then I was like omg and he was like no way', 'and I ran out.']`

Part 2: Tokenization

1. **Question:** Suggest a simplistic top-down approach for tokenization in English text. Identify specific situations where your proposed approach may fail to produce expected results. [1 mark]

   Answer: Firstly, we must clarify what a word means. According to [7], this task is actually very complex, with the following points to be considered:

   - A word is usually classified based on orthographic means, which are based on the writing system and are usually separated by spaces. However, this can be misleading; for example, "I'm" is one orthographic word but functions grammatically as two words. Such words are called clitics.
   - A combination of words separated by a hyphen is considered as one word.
   - Depending on the application, counting words might include punctuation and spoken buffers also.
   - Words are also generally units of meaning, as discussed in class.

   Now, a token is the output of a tokenization algorithm [6]. It does not always end up carrying meaning, and is composed of the following categories of units:

   - Words
   - Subword/Morpheme, which are smaller meaning-bearing chunks
   - Individual characters

   A rule-based approach to splitting the corpus into tokens is proposed below:

   - Split the entire corpus based on whitespaces into words.
   - Punctuation (such as `[].,;"'?():_-`) is counted as a token, but is a separate token (not considered jointly with the word it follows).

   This system will face challenges with the following tokens:

   - Clitics will be treated as fragmented words separated by an isolated apostrophe, giving three meaningless tokens.
   - Abbreviations that rely on periods will be damaged, such as `Ph.D.`. Similarly, decimals, dates, E-mail IDs, and URLs will be affected.

2. **Question:** Study about NLTK's Penn Treebank tokenizer and spaCy's tokenizer. What type of knowledge does each of them use Top-down, Bottom-up, or a hybrid approach? Clearly justify your answer for both tokenizers. [2 marks]

   Answer: The Penn Treebank tokenizer uses regular expression rules created during the Penn Treebank project. Hence it uses a more elaborate top-down approach than the one proposed earlier. These rules are:

   - In general, the splitting of words is whitespace-driven.

- Punctuation is broken into their own tokens. But periods inside abbreviations and decimals are not split.
- Hyphenated words are kept together
- All clitics are split into their base words and affixes.

The spaCy tokenizer, however, first uses the top-down classical rule of splitting based on whitespace [11]. Then, it uses a hardcoded collection of exceptions specific to the target use-case or language, for example the handling of clitics, similar to a dataset collected from bottom-up methods. Once that is done, a collection of top-down regex rules based on prefixes, suffixes, and infixes are applied. Hence, the spaCy tokenizer is a hybrid tokenizer.

3. **Question:** Perform word tokenization of the sentence-segmented documents using: [4 marks]

   (a) The proposed top-down method

   (b) Penn Treebank Tokenizer

   (c) spaCy Tokenizer

   State a possible scenario along with an example where:

   (a) Your approach performs better than the Penn Treebank and spaCy tokenizers

   (b) Your approach performs worse than the Penn Treebank and spaCy tokenizers

   Answer: The tokenization of the sentences obtained by sentence segmentation is completed using the provided code.

   (a): The naive approach performs better when processing messy data, or mathematical/code equations with operators, in some domain-specific use-cases. For example, parsing a mathematical equation, such as `3+5=7`. The naive approach extracts all the involved numbers and operations in the equation - `['3', '+', '5', '=', '7']`. The Penn Treebank standard keeps the entire equation together. spaCy on the other hand does not split `5=7` due to it being one of the hardcoded cases. Thus, Penn Treebank and spaCy might find difficulty in usage in very specific use-cases and give non-intuitive results.

   (b): The naive approach performs drastically worse when parsing formal English text containing clitics and decimals, because it blindly divides meaning-bearing symbols. For example, in the sentence `She doesn't have $5.00.`, the naive approach will fragment the sentence into `['She', 'doesn', "'", 't', 'have', '$', '5', '.', '00', '.']`. Penn Treebank and spaCy separate clitics logically (turning `doesn't` into `does` + `n't`) and keep prices intact (preserving `.00` as a single token).

   One interesting observation I made while running test cases is that since the regex behind Penn Treebank was developed in the late 80s and 90s using standard ASCII characters, using variations of characters expected by the regex, such as keyboard apostrophe (') rather than Unicode curly quote (’), will lead to unexpected results.

Part 3: Stemming and Lemmatization

1. **Question:** What is the difference between stemming and lemmatization? Clearly explain the conceptual differences between the two approaches in terms of linguistic knowledge, output form, and computational complexity. Give suitable examples to illustrate your answer. [1 mark]

   Answer: The tokens obtained after sentence segmentation and tokenization may correspond to tokens of different morphology but of the same base or root form. As seen in class, such variations in word form are obtained by inflection or derivation. Keeping these different forms is problematic because if the system recognizes them as different words then all downstream processing will provide wrong answers, such as frequency calculation.

   Stemming solves this by using a top-down set of rules to chop off common affixes from the ends of words (such as stripping "-ing," "-ly," or "-es") and produces a base form known as a "stem". It does not operate with linguistic knowledge of the word and so the stem is often heavily truncated and may not be a valid, real dictionary word. For example, crudely reducing "studies" to "studi". But because stemming only relies on a blind execution of rules, its computational complexity is relatively low. Stemming is applied to the reduction of both inflection and derivation.

   The Porter stemmer is used in this assignment [2, 8]. It calculates a value known as measure of the word. The set of rule steps that the Porter stemmer uses is:

   - Removes basic plural suffixes and past/present participles. For example, it converts `SSES` $\rightarrow$ `SS`, `IES` $\rightarrow$ `I`, and removes `ING` or `ED` if the preceding part of the word contains a vowel.
   - Maps multi-syllable suffixes to simpler ones, provided the root word has enough measure. For example, it maps `ATIONAL` $\rightarrow$ `ATE`, and `IZATION` $\rightarrow$ `IZE`.
   - Further simplifies suffixes to their base forms. For example, it converts `ICAL` $\rightarrow$ `IC`, and entirely removes suffixes like `FUL` and `NESS`.
   - Strips off a long, specific list of suffixes (such as `AL`, `ANCE`, `ER`, `IC`, `ABLE`, `MENT`) strictly if the remaining stem has a measure greater than 1.
   - Cleans up the final string by removing a trailing `e` (if the word measure is long enough to support it) and reducing double consonants at the end of a word (like `LL`) to a single consonant (`L`).

   Lemmatization utilizes deeper linguistic knowledge (using WordNet, for example) to conduct a proper morphological analysis of the text. It does Part-of-Speech tagging, looks at the word context, and then uses a database to find out how to properly strip affixes to produce a sensible base form, called a lemma. The lemma is guaranteed to be a structurally correct, valid word in the language. It resolves complex grammatical rules and irregular word forms (such as correctly mapping the word "better" to its root "good") and so the computational complexity and time required is higher than stemming. Lemmatization is also more focused on reducing inflections.

   The WordNet lemmatizer uses an algorithm called Morphy that does these steps [9, 3]:

- The lemmatizer assigns a Part-of-Speech (POS) tag. To work accurately, it needs to know if the word is acting as a Noun, Verb, Adjective, or Adverb.
- It first searches the massive WordNet database. If the exact word is already found in the dictionary under the provided POS, it returns the word exactly as is.
- Before doing any string manipulation, it checks a hardcoded list of known irregular words. If the word is an irregular exception (e.g., `mice` → `mouse`, or `taught` → `teach`), it immediately returns the mapped dictionary lemma.
- If the word is not found in the dictionary or exception lists, it applies specific suffix-stripping rules tailored to the POS. For a noun, it will try removing `-s`, `-es`, or `-ies`. For a verb, it tries removing `-ed` or `-ing`.
- After stripping a suffix in Rule 4, it takes the newly generated string and searches the dictionary again. It only accepts and returns this new string if it is successfully validated as a real, existing word in the WordNet database.

Some examples to show the difference in the two algorithms are given below:

- Original Word: `studies`
  Porter Stemmer Output: `studi`
  WordNet Lemmatizer Output: `study`.
- Original Word: `better`
  Porter Stemmer Output: `better`.
  WordNet Lemmatizer Output: `good`.
- Original Word: `leaves`
  Porter Stemmer Output: `leav`.
  WordNet Lemmatizer Output: `leaf`.

2. **Question:** Using the word-tokenized text from the Cranfield Dataset: [3 marks]

   (a) Perform stemming using Porter's Stemmer

   (b) Perform lemmatization using WordNet Lemmatizer

   Compare the outputs of the two methods by analyzing:

   - Changes in vocabulary size
   - Examples of over-stemming (if any)
   - Cases where lemmatization preserves semantic meaning better

   Answer: The 2 methods have been implemented in the given code templates. Before reduction, the vocabulary size was 8372, and after reduction, the results are:

   - Stemming - 5735
   - Lemmatization - 6996

   After observing the outputs, it is clear that both methods help reduce the vocabulary size, with stemming performing better at absolute vocabulary reduction. However, stemming produces several new stems that are not words and that also could have been reduced to the same base

form as other words. Lematization takes care of this. For example, both `good` and `better` map onto the same lemma, `good`, but to different stems, `good` and `better` respectively.

Worse, stemming sometimes results in 2 unrelated words happening to have the same stem, which lemmatization prevents. For example, the stems of `his` and of `hi` is `hi`, but both have different lemma.

This is one such case of over-stemming, resulting in either unrelated words having the same stem or of a stem completely losing meaning. Other examples from the corpus are:

- `experimental` → `experiment`
- `investigation` → `investig`
- `aerodynamics` → `aerodynam`
- `propeller` → `propel`
- `determine` → `determin`

In general, stemming results in common over-stemming cases such as: `systematically` → `systemat`, `organizations` → `organ`, `organizes` → `organ`, `because` → `becaus`

Stemming also leaves some words untouched, which lemmatization brings to its true root, such as:

- `made` → `make`
- `were` → `be`
- `as` → `a`
- `found` → `find`
- `is` → `be`

Clearly, given the errors above, lemmatization maintains the original meaning of the sentence while merging common bases far more accurately than stemming. Stemming will prove to be disastrous when the corpus size increases, because we will have several new nonsensical words and also will end up merging unrelated words through their base forms, resulting in both a lack of meaning and a dilution of the sentence meaning. Lemma may still preserve sentence meaning, unlike the corresponding stems.

Part 4: Stopword Removal

1. **Question:** Remove stopwords from the tokenized documents using a curated list, such as the stopword list provided by NLTK. [1 mark]

   Answer: The `stopwords` collection in NLTK contains 198 pre-defined stopwords that are commonly used in English and are a closed vocabulary [10]. These stopwords are useful for most general purpose text, though the actual definition of a stopword depends on the use-case.

   The code implements this top-down approach after downloading the set, as in the function `fromList` in the class `StopwordRemoval`.

2. **Question:** Suggest a bottom-up (data-driven) approach for constructing a stopword list that is specific to the given corpus of documents.

   Clearly explain the intuition behind your proposed method. [1 mark]

   Answer: To do this, I first specify what a stopword means in a general corpus with the following points:

   - A stopword is a commonly occuring word in the corpus that does not have semantic meaning but is used to connect words in a sentence.
   - Stopwords are a closed vocabulary in the langauge and are still the most commonly used words in the language.

   Hence, a good way to identify stopwords is to rank order the words in the tokenized corpus based on their term frequency, and then choose the top `k` ranks, where `k` is a parameter.

   The natural next question is the process of choosing `k`. I derive inspiration from Zipf's law [5], which connects the value of the rank of a term to the value of the frequency of the term - the frequency of a word is proportional to its rank raised to a negative power.

   Hence, instead of setting a rank threshold, we can set a frequency threshold and choose the words that are shortlisted in this manner as our stopwords [1, 12]. This is more useful because statistical significance for frequency is more semantically useful - saying that a word is at rank 1 in a corpus is not as useful as saying that it has occurred 15000 times in a 1400-document corpus.

   To identify stopwords, I recognize them as statistical outliers and use the `mean + 3 * standard deviation` of the term frequency distribution as the cutoff measure. This value for the given corpus is `198.08`, giving me `168` stopwords.

   This value of the cutoff, as seen from the graph attached, also captures words that are so frequent that their frequency occurs before the curve flatlines.

   A point to note is that the NLTK stopwords contain both clitics as well as affixes of these clitics, such as `we'll` and `'ll`. The custom stopword set was made after tokenization and before inflection reduction was completed, so that cliques could be included in the set.

   Another point is that the Cranfield dataset is a foundational, small-scale information retrieval (IR) test collection consisting of 1,400 documents (abstracts) in aerodynamics, along with 225

queries and relevance judgments (qrels). It is hence much more specific than the corpus that NLTK stopwords is based on. This will be shown in the answer for the next question.

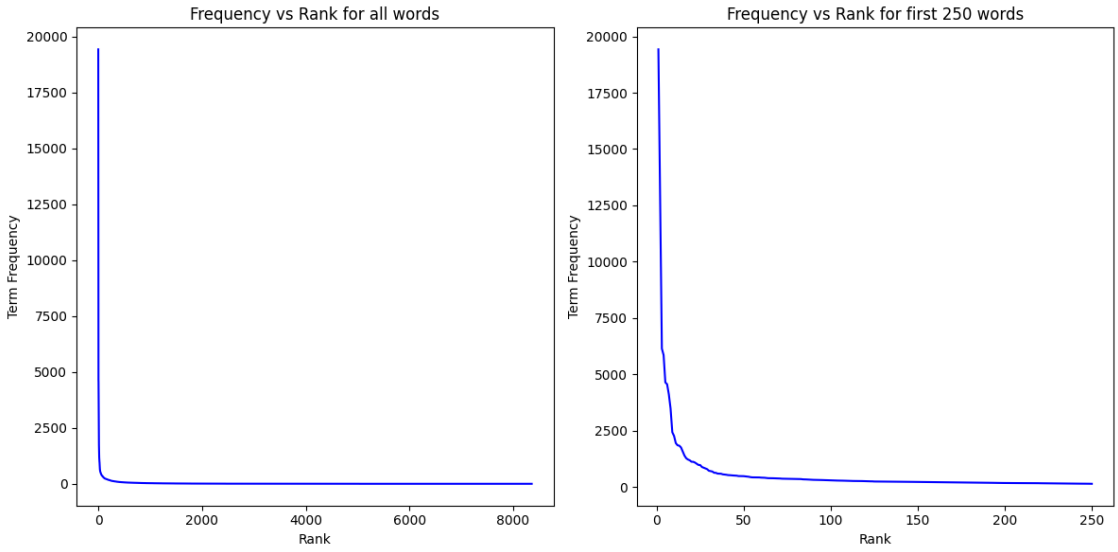

Figure 1: Zipf's Law distribution showing Term Frequency versus Rank on the Cranfield dataset.

3. **Question:** Implement the strategy proposed in the previous question and compare the stopwords obtained with those provided by NLTK on the Cranfield dataset. [2 marks] Your comparison should include:

   - Number of stopwords identified
   - Overlap between the two lists
   - Words that appear in one list but not the other

   Answer: We use the custom method mentioned above to generate our own list of stopwords. We compare the 2 sets of stopwords in the criteria mentioned above.

   The NLTK stopword collection has the following elements in it:

   "it'll", 'such', "that'll", 'has', 'at', 'it', 'should', "you'll", "mightn't", 'not', 'of', 'his', 'needn', "needn't", "she'll", 'ourselves', 'wasn', 've', 'where', "haven't", "couldn't", "shan't", "they'd", 'under', 'why', 'have', 'having', 'for', "we've", 'weren', 'through', 'your', "aren't", "i've", 'did', "isn't", 't', 'between', 'myself', "they're", 'had', 'above', 'do', 'once', 'she', "it'd", 'same', 'with', 'on', 'am', 'herself', "should've", 'whom', 'be', "she's", "he'll", 'now', 'to', "hasn't", "mustn't", "we'd", 'm', 'a', "didn't", 'its', 'will', 'so', 'haven', 'are', 'me', "he'd", 'nor', "we're", 'y', "hadn't", 'out', "you're", 'our', 'does', 'up', 'into', 'i', 'each', 'you', 'this', 'doesn', 'own', 'hadn', "won't", 'during', 'mustn', 'by', 'because', 'her', 'from', "don't", 's', 'we', 'him', 'no', 'very', 'won', 'and', 'ma', 'while', 'only', 'shan', 'most', 'being', 'about', 'them', 'he', 're', "we'll", 'until', 'aren', 'ain', 'which', 'as', 'itself', 'these', 'further', 'some', "she'd", "i'd", 'all', 'been', 'if', "wouldn't", 'both', "you'd",

'were', 'but', 'other', 'mightn', 'what', 'here', 'against', 'yours', 'couldn', 'just', 'themselves', 'wouldn', 'can', 'hasn', 'those', "weren't", 'd', "you've", 'himself', 'didn', "shouldn't", 'before', 'yourself', 'll', 'my', 'shouldn', 'again', "i'll", "it's", 'who', 'more', 'or', 'down', 'how', 'hers', 'over', 'theirs', 'few', 'when', 'there', 'any', 'then', "they've", 'after', "i'm", 'an', 'don', 'too', 'is', 'ours', 'o', 'their', "he's", 'below', 'in', 'off', "wasn't", 'they', "they'll", 'was', 'isn', 'the', 'than', "doesn't", 'doing', 'that', 'yourselves'

Using the bottom-up approach to obtain a list of stopwords, we get the following stopwords:

'such', 'fluid', 'has', 'at', 'it', 'flight', 'not', 'of', 'point', 'measurements', 'approximate', 'where', 'based', 'general', 'under', 'have', 'equations', 'high', 'for', 'region', 'constant', 'theoretical', 'numbers', 'dimensional', 'transition', 'numerical', 'investigation', 'layer', 'made', 'between', 'cylinder', 'shock', 'with', 'on', 'body', 'heat', '1', 'be', 'to', 'laminar', 'tunnel', 'a', "'s", 'also', 'are', 'data', 'experimental', 'values', 'results', 'first', 'order', 'into', 'mach', 'analysis', 'model', 'this', 'found', 'flat', 'lift', 'wings', 'transfer', 'wave', 'low', 'by', 'considered', 'value', 'from', 'case', 'small', 'temperature', 'velocity', 'use', 'thin', 'and', 'only', 'methods', 'flutter', 'about', 'stress', 'gas', 'leading', 'tests', 'problems', 'which', 'as', 'these', 'may', 'surface', 'flows', 'some', 'compared', 'shown', 'been', 'buckling', 'one', 'all', 'form', 'both', 'were', 'ratio', 'but', 'other', 'paper', 'jet', 'hypersonic', 'can', 'large', 'agreement', 'field', 'boundary', 'thickness', 'used', 'stagnation', 'effect', 'separation', 'plate', 'air', 'effects', 'aerodynamic', 'flow', 'method', 'speeds', 'or', 'wing', 'over', 'when', 'number', 'distribution', 'stability', 'angle', 'stream', 'presented', 'speed', 'characteristics', 'an', 'skin', 'drag', 'obtained', 'conditions', 'solutions', 'is', 'present', 'circular', 'function', 'free', 'problem', 'discussed', 'equation', 'pressure', 'two', 'in', 'reynolds', 'edge', 'attack', 'supersonic', 'was', 'theory', 'the', 'wall', 'bodies', 'simple', 'than', 'given', 'solution', 'turbulent', 'range', 'that', 'distributions'

There are 198 NLTK stopwords and 168 custom stopwords identified.

Naturally, the words common to the 2 sets are the ones that are relevant to maintaining the structure of sentences. There are 45 such words, and they are given by:

'such', 'about', 'or', 'has', 'to', 'at', 'it', 'over', 'a', 'when', 'not', 'which', 'of', 'as', 'these', 'are', 'some', 'where', 'been', 'all', 'an', 'both', 'into', 'were', 'but', 'other', 'is', 'under', 'have', 'this', 'for', 'in', 'can', 'was', 'the', 'by', 'between', 'from', 'than', 'with', 'on', 'that', 'and', 'only', 'be'

There are 153 words present in NLTK but not in the custom set, and these words are predominantly clitics and their derivatives.

"it'll", "that'll", 'should', "you'll", "mightn't", 'needn', 'his', "needn't", "she'll", 'ourselves', 'wasn', 've', "haven't", "couldn't", "shan't", "they'd", 'why', 'having', "we've", 'weren', 'through', 'your', "aren't", "i've", 'did', "isn't", 't', 'myself', "they're", 'had', 'above', 'do', 'once', 'she', "it'd", 'same', 'am', 'herself', "should've", 'whom', "she's", "he'll", 'now', "hasn't", "mustn't", "we'd", 'm', "didn't", 'its', 'will', 'so', 'haven', 'me', "he'd", 'nor', "we're", 'y', "hadn't", 'out', "you're", 'our', 'yourselves', 'does', 'up',

```
'i', 'each', 'you', 'doesn', 'own', 'hadn', "won't", 'during', 'mustn', 'her',
'because', 's', "don't", 'we', 'him', 'no', 'very', 'won', 'ma', 'while', 'shan',
'most', 'being', 'them', 'he', 're', "we'll", 'until', 'aren', 'ain', 'itself',
'further', "she'd", "i'd", 'if', "wouldn't", "you'd", 'mightn', 'what', 'here',
'against', 'yours', 'couldn', 'just', 'themselves', 'wouldn', 'hasn', 'those',
"weren't", 'd', "you've", 'himself', 'didn', "shouldn't", 'before', 'yourself',
'll', 'my', 'shouldn', 'again', "i'll", "it's", 'who', 'more', 'down', 'how',
'hers', 'theirs', 'few', 'there', 'any', 'then', "they've", 'after', "i'm", 'don',
'too', 'ours', 'o', 'their', "he's", 'below', 'off', "wasn't", 'they', "they'll",
'isn', "doesn't", 'doing'
```

There are 123 words only present in the custom set, and these words are very domain/corpus specific, and as the Cranfield dataset is aerospace-related, the words are relevant to this theme. This shows us that the words we deem as irrelevant or common depends heavily on the corpus in which these words are present.

```
'fluid', 'flight', 'point', 'measurements', 'approximate', 'based', 'general',
'equations', 'high', 'region', 'constant', 'theoretical', 'numbers', 'transition',
'dimensional', 'numerical', 'investigation', 'layer', 'made', 'cylinder', 'shock',
'body', '1', 'heat', 'laminar', 'tunnel', "'s", 'also', 'data', 'experimental',
'values', 'results', 'first', 'order', 'mach', 'analysis', 'model', 'found', 'flat',
'lift', 'wings', 'transfer', 'wave', 'low', 'value', 'considered', 'case', 'small',
'temperature', 'velocity', 'use', 'thin', 'methods', 'flutter', 'stress', 'gas',
'leading', 'tests', 'problems', 'may', 'surface', 'flows', 'compared', 'shown',
'buckling', 'one', 'form', 'ratio', 'paper', 'jet', 'hypersonic', 'large', 'agreement',
'field', 'boundary', 'thickness', 'used', 'stagnation', 'effect', 'separation',
'plate', 'aerodynamic', 'effects', 'flow', 'method', 'speeds', 'wing', 'range',
'number', 'distribution', 'stability', 'angle', 'stream', 'presented', 'speed',
'characteristics', 'skin', 'drag', 'obtained', 'conditions', 'solutions', 'present',
'circular', 'function', 'free', 'problem', 'discussed', 'equation', 'pressure',
'two', 'reynolds', 'edge', 'attack', 'supersonic', 'theory', 'wall', 'bodies',
'simple', 'given', 'solution', 'turbulent', 'air', 'distributions'
```

Note that if the user is worried about the semantic relevance of the words being set as stopwords, then the following things can be done:

- Use a larger frequency cutoff and shortlist fewer words, in which case, as noted above, the intersection of the custom stopword list and the NLTK stopword list will increase and the custom list will have less semantically relevant words.
- Use an external larger dataset which may be relevant to the Cranfield dataset, obtain stopwords from that, and then use that list in this dataset. This may cause problems about which words occur frequently in the Cranfield dataset and which ones occur frequently in the larger reference dataset.
- Combine the term frequency metric with Document Frequency (DF) or Part-of-Speech (POS) tagging.

Part 5: Retrieval

1. **Question:** Given a set of queries Q and a corpus of documents D, what would be the total number of similarity computations required to estimate the similarity between every query and every document?

   Assume that TF-IDF vector representations are already available for both queries and documents over a vocabulary V. Clearly express your answer in terms of Q, D, and V. [1 mark]

   Answer:

   To estimate the similarity between every query and every document using a naive approach, we must compute the similarity score for every possible query-document pair. Given $|Q|$ queries and $|D|$ documents, there are exactly $|Q| \cdot |D|$ pairwise vector comparisons to evaluate. We compute the Term Frequency for a given document, and compute the Inverse Document Frequency for a given term. The IDF calculation is done D times and the TF calculation is done $D \cdot V$ times. This is repeated for every query.

   Therefore, the total number of arithmetic computations required is proportional to the product of these three dimensions, resulting in $\mathcal{O}(|Q| \cdot |D| \cdot |V|)$.

2. **Question:** Suggest how the concept of an inverted index can be used to reduce the time complexity derived in the previous question. You may introduce additional variables (such as average document frequency or sparsity factors) to explain your reasoning. Provide a clear comparison between the naive approach and the inverted-index-based approach in terms of computational complexity. [2 marks]

   Answer: An Inverted Index drastically reduces this time complexity by exploiting the extreme sparsity of natural language TF-IDF vectors. Instead of blindly comparing a query vector to every document vector, an inverted index maps each term in the vocabulary directly to a postings list of only the specific documents that actually contain that term.

   Let us introduce two sparsity variables below:

   - L_Q: The average number of unique non-zero terms in a given query (where $L_Q \ll |V|$, since the query in practice is small).
   - P: The average length of a postings list, representing the average number of documents that contain a specific term (where $P \ll |D|$, since, as seen in Zipf's law and the assumptions of the previous questions, non-stopwords are used selectively and sparingly).

   For each query, we first identify only the L_Q terms actually present in the query string. For each of those terms, it fetches the postings list and processes only the P documents containing the term. Thus, the time complexity drops significantly to $\mathcal{O}(|Q| \cdot L_Q \cdot P)$, much smaller than the forward index's $\mathcal{O}(|Q| \cdot |D| \cdot |V|)$.

   It effectively transforms an inefficient $\mathcal{O}(N^3)$ exhaustive search into a highly optimized sparse matrix multiplication.

References

[1] Kavita Ganesan. Tips for constructing custom stop word lists. https://kavita-ganesan.com/tips-for-constructing-custom-stop-word-lists/. Accessed: 2026-03-16.

[2] GeeksforGeeks. Porter stemmer technique in natural language processing. https://www.geeksforgeeks.org/nlp/porter-stemmer-technique-in-natural-language-processing/. Accessed: 2026-03-16.

[3] GeeksforGeeks. Python — lemmatization with nltk. https://www.geeksforgeeks.org/python/python-lemmatization-with-nltk/. Accessed: 2026-03-16.

[4] GeeksforGeeks. Python — perform sentence segmentation using spacy. https://www.geeksforgeeks.org/python/python-perform-sentence-segmentation-using-spacy/. Accessed: 2026-03-16.

[5] GeeksforGeeks. Zipf's law. https://www.geeksforgeeks.org/nlp/zipfs-law/. Accessed: 2026-03-16.

[6] Data Science in your pocket. Tokenization algorithms in natural language processing (nlp). https://medium.com/data-science-in-your-pocket/tokenization-algorithms-in-natural-language-processing-nlp-1fceab8454af. Accessed: 2026-03-16.

[7] Daniel Jurafsky and James H. Martin. *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition*. Pearson, 2nd edition, 2009.

[8] Vijini Mallawaarachchi. Porter stemming algorithm. https://vijinimallawaarachchi.com/2017/05/09/porter-stemming-algorithm/, 2017. Accessed: 2026-03-16.

[9] NLTK. nltk.stem.wordnetlemmatizer. https://www.nltk.org/api/nltk.stem.WordNetLemmatizer.html. Accessed: 2026-03-16.

[10] PythonSpot. Nltk stop words. https://pythonspot.com/nltk-stop-words/. Accessed: 2026-03-16.

[11] spaCy. Linguistic features: How the tokenizer works. https://spacy.io/usage/linguistic-features#how-tokenizer-works. Accessed: 2026-03-16.

[12] Stack Overflow. Can stop words be found automatically? https://stackoverflow.com/questions/22370144/can-stop-words-be-found-automatically. Accessed: 2026-03-16.